

Prompting GPT and Codex for Reliable Results

Executive summary

The current OpenAI guidance points to a clear shift in how to write prompts for modern GPT and Codex systems. For **current general-purpose GPT work**, OpenAI now recommends **shorter, outcome-first prompts** that define what success looks like, what constraints matter, what evidence is available, and what the final answer should contain. For **agentic coding with Codex**, the best prompts are more operational: they specify autonomy, persistence, tool-use rules, verification steps, and what “done” means, while avoiding unnecessary rollout chatter that can cause early stopping in some Codex flows. ¹

Across model generations, the common denominator is no longer “make the prompt longer.” The durable pattern is: **pick the right model and reasoning setting; make the task contract explicit; constrain outputs with schemas or clear formatting; add examples only when zero-shot behavior is unstable; ground the model with targeted retrieval rather than dumping context; and measure every meaningful prompt change with evals and traces**. OpenAI’s current docs repeatedly emphasize prompts-as-code, eval-driven iteration, structured outputs, and high-signal observability. ²

The strongest practical takeaway is that **prompt engineering is now inseparable from workflow engineering**. In production, a “good prompt” is usually a combination of: a prompt contract, the right tool surfaces, a retrieval strategy, a schema, tests, graders, and deployment discipline. Academic work still supports techniques like chain-of-thought, self-consistency, least-to-most decomposition, and few-shot prompting for hard reasoning problems, but OpenAI’s latest GPT-5.5 guidance warns that older, process-heavy prompt stacks can now add noise or make outputs feel mechanical. ³

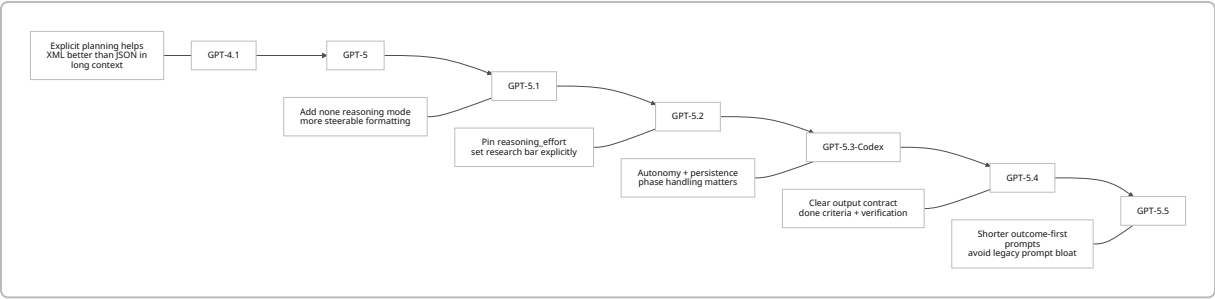
Scope and version assumptions

As of **June 19, 2026**, OpenAI’s model docs recommend **GPT-5.5** as the default starting point for complex reasoning and coding, with **GPT-5.4 mini** and **GPT-5.4 nano** positioned as faster, cheaper variants for lighter workloads. The current docs also show that GPT-5.5, GPT-5.4, and GPT-5.4 mini all support long contexts and hosted tools such as functions, web search, file search, and computer use. ⁴

For **Codex specifically**, availability differs by surface. The **Codex product docs** currently recommend **gpt-5.5** for most tasks and say that **gpt-5.2** and **gpt-5.3-codex** are deprecated as user-selectable models in Codex for users signed in with ChatGPT, while noting that some deprecated ChatGPT-surface models may still be available through the API with API-key authentication. At the same time, the **Codex prompting guide** is explicitly written for `gpt-5.3-codex` and says that, in the API, the Codex-tuned model is `gpt-5.3-codex`. The practical inference is that **gpt-5.3-codex guidance remains directly relevant for API and harness design, while gpt-5.5 is the current default for the Codex product experience**. ⁵

Older versions still matter because some behaviors are stable across generations. OpenAI’s GPT-5.2 and GPT-5.1 guidance largely carries forward to later GPT-5.x models for ideas like prompt migration, reasoning-

effort pinning, tool descriptions, explicit output contracts, and eval-driven iteration. GPT-4.1 remains useful as an older-but-still-relevant reference for **literal instruction following, long-context prompt placement, delimiter choice, and common failure modes**. 6



The timeline above is a synthesis of OpenAI's prompt guides and model docs. 7

Best-practice catalog

The table below prioritizes techniques that are both **officially recommended** and **consistently useful in production**.

Practice	Why it works	Typical prompt pattern	Trade-offs	Example prompt fragment	Ev
Write outcome-first task contracts	GPT-5.5 performs best when the prompt defines the desired outcome, constraints, evidence, and deliverable, rather than prescribing every micro-step.	Goal + Success criteria + Constraints + Output + Stop rules	If too short, the prompt can become under-specified; if too procedural, GPT-5.5 can become mechanical.	"Produce a concise incident report. Use only the attached logs. Include root cause, confidence, and next actions. Stop when you have enough evidence."	8

Practice	Why it works	Typical prompt pattern	Trade-offs	Example prompt fragment	Ev
Keep personality and collaboration style short	OpenAI now distinguishes how the assistant should <i>sound</i> from how it should <i>work</i> ; both help steer UX without replacing the task contract.	Short <code># Personality</code> and <code># Collaboration style</code> blocks	Overlong persona text often crowds out more important task rules.	"Be direct, respectful, and practical. Make progress with reasonable assumptions; ask only when missing data would materially change the result."	9
Use explicit output contracts or schemas	Structured Outputs can enforce JSON Schema, reduce retries, and make refusals and downstream parsing easier to handle.	JSON Schema, fixed section names, explicit length/format limits	Strong schemas reduce flexibility; for narrative tasks they can feel rigid.	"Return JSON with keys: <code>answer</code> , <code>evidence</code> , <code>uncertainty</code> , <code>next_step</code> ."	10
Add few-shot examples only when zero-shot is unstable	Few-shot still works well for steering new tasks and calibration, especially classification or style transfer. Diverse examples are recommended.	<code># Examples</code> with representative input/output pairs	Examples increase token cost and can overfit phrasing if too narrow or repetitive.	"Examples: short reviews → one-word sentiment labels."	11

Practice	Why it works	Typical prompt pattern	Trade-offs	Example prompt fragment	Ev
Ground with relevant context, not maximal context	Retrieval improves factuality and scope control; file search and web search are first-class tools in OpenAI's stack.	<code>Context</code> block or tool access plus scope rule such as "use only provided sources unless noted"	Dumping too much context degrades retrieval and reasoning, especially in long contexts.	"Use the uploaded design doc and ticket history as primary evidence. If evidence is missing, say so explicitly."	12
Format prompts with Markdown sections and XML boundaries	Markdown improves readability and hierarchy; XML helps isolate context segments. GPT-4.1 long-context testing found XML-like formats stronger than JSON for document delimiting.	<code># Identity,</code> <code>#</code> <code>Instructions,</code> <code># Examples,</code> <code><doc id=...></code>	XML is extra ceremony for simple prompts; JSON can still be fine for coding/data contexts.	"Wrap each retrieved document in <code><doc></code> tags and refer to document IDs in the answer."	13
Prefer decision rules over blanket absolutes	OpenAI cautions that strict <code>ALWAYS</code> / <code>NEVER</code> rules should be reserved for true invariants. Decision rules work better for tool use, clarification, and iteration.	Replace rigid commands with criteria and stop conditions	Too much flexibility can increase variance if your criteria are vague.	"Use the fewest useful tool loops, but do not let loop minimization outrank correctness or citations."	14

Practice	Why it works	Typical prompt pattern	Trade-offs	Example prompt fragment	Ev
Escalate to decomposition only when the task truly benefits	Chain-of-thought, self-consistency, and least-to-most prompting improve hard reasoning in the literature, but GPT-5.5 often needs less process scaffolding than earlier models.	"First decompose the problem into subproblems..." or use multi-sample judging in evals	Improves difficult reasoning, but increases cost, latency, and sometimes verbosity.	"Break this migration into subproblems, solve each, then synthesize the final change plan."	15
Treat prompts as code and version them	OpenAI now recommends code-managed prompts with typed parameters, tests, PR review, and staged rollout rather than reusable prompt objects.	Prompt builders in code, typed args, fixtures, feature flags	Requires engineering discipline, but makes behavior auditable and rollback-safe.	<code>buildSupportReplyPrompt({policy, locale, customerTier})</code>	16

For **Codex and coding-agent workflows**, the most important patterns are more operational than conversational.

Codex-specific pattern	Why it works	Typical harness/prompt pattern	Trade-offs	Example prompt fragment	Evidence
Autonomy + persistence + bias to action	OpenAI's Codex guide recommends framing the model as an autonomous engineer that keeps going through context gathering, implementation, testing, and refinement.	"Gather context, implement, test, refine, then report results."	Can overrun scope unless you also define boundaries and stop rules.	"Diagnose, patch, run the smallest relevant tests, and summarize outcomes without waiting for step-by-step approval."	17
Explicit tool hierarchy	Codex performs better when tool choices are crisp and semantically correct; use dedicated tools before shell fallbacks.	Prefer <code>read_file</code> , <code>apply_patch</code> , <code>rg</code> , git-specific tools	Requires a well-designed tool surface.	"Use dedicated repo tools before raw shell; use terminal only if no dedicated tool can perform the action."	18
Parallelize repo exploration	OpenAI specifically recommends batching reads and using parallel tool calls for multi-file exploration.	Plan all likely reads, run one parallel batch, then analyze	Can complicate debugging if results are noisy; sequential reads still matter when each step depends on the previous one.	"Read the caller, callee, failing test, and config files in one batch before proposing the fix."	19

Codex-specific pattern	Why it works	Typical harness/prompt pattern	Trade-offs	Example prompt fragment	Evidence
Require verification, not just patches	Codex docs and community practice both emphasize that tests define “done” and constrain behavior.	“After the fix, run lint + the smallest relevant test suite; report commands and results.”	Verification costs time and tokens, but usually saves downstream review time.	“Fix the failure, rerun the repro, then run the smallest relevant tests and report the evidence.”	20
Avoid unnecessary rollout preambles in gpt-5.3-codex flows	The Codex guide explicitly says to remove upfront plans or preambles during rollout because they can cause abrupt stopping; if you do use preambles, phase handling matters.	Final result-oriented behavior instead of chatty progress narration	Less transparency unless your UI handles phases or traces separately.	“Do the work first; report once you have a verified result. Do not narrate every step unless blocked.”	21
Use skills for recurring workflows	Skills package instructions, resources, and scripts with progressive disclosure, which improves reuse without shoving all rules into every prompt.	Small role-like skills with clear descriptions and trigger words	Community feedback notes speed/token overhead if skills become omnibus policy bundles.	“\$backend-auditor review this API diff for auth regressions and missing tests.”	22

Codex-specific pattern	Why it works	Typical harness/prompt pattern	Trade-offs	Example prompt fragment	Evidence
Be concrete on frontend constraints	For frontend tasks, OpenAI recommends specific libraries and concrete design constraints rather than generic “make it nice” prompts.	State framework, styling libs, fidelity rules, responsive behavior	More specificity can reduce creative exploration.	“Recreate this screenshot in React + Vite + Tailwind, matching spacing, typography, and mobile behavior closely.”	23

The following synthesized templates align with the official guidance above rather than copying it verbatim.

24

Sample prompt for a general GPT task

Role: You are a policy analyst helping an operations team make fast, defensible decisions.

Personality

Be direct, calm, and practical.

Goal

Decide whether the request is allowed under the attached policy.

Success criteria

Use only the supplied policy text.

If the policy is ambiguous, explain the ambiguity instead of guessing.

Return a final answer that a manager could act on immediately.

Constraints

Do not use outside knowledge.

Quote or cite the exact policy section for every material claim.

Output

Return:

1. Decision
2. Evidence
3. Uncertainty
4. Recommended next step

Stop rules

If the policy is missing the needed rule, say exactly what is missing and stop.

Sample prompt for a Codex bug-fix task

You are a senior software engineer working in this repo.

Goal: diagnose and fix the bug causing settings to appear saved without persisting.

Success criteria:

- identify the root cause
- implement the smallest correct fix
- rerun the repro steps
- run lint plus the smallest relevant test suite
- report the commands run and the results

Constraints:

- preserve existing architecture and naming conventions
- avoid unrelated refactors
- prefer dedicated repo tools over raw shell when available
- read all likely relevant files in parallel before editing

Stop rules:

- if a missing credential, secret, or external dependency blocks verification, say exactly what blocked you
- otherwise do not stop at analysis; carry through to a verified result

Version-specific guidance

Model or family	What changed for prompting	What to emphasize	What to avoid	Practical recommendation
GPT-5.5	Shorter, outcome-first prompts generally outperform older process-heavy prompt stacks; low/medium reasoning should be re-evaluated before escalating.	Outcome, constraints, evidence, final artifact, short personality/ collaboration blocks, decision rules, stop rules.	Dragging forward legacy mega-prompts from older stacks.	Start simple, add structure only where evals show instability. ²⁵

Model or family	What changed for prompting	What to emphasize	What to avoid	Practical recommendation
GPT-5.4	Stronger long-running task performance, better control over style and output contracts, more disciplined verification loops.	Explicit output contracts, citation rules, tool expectations, completion criteria.	Vague definitions of “done.”	Use when you need production-grade multi-step execution and evidence-rich answers. ²⁶
GPT-5.3-Codex	Codex-specific harness tuning matters: autonomy, repo exploration, tool use, parallel reads, and phase-aware handling for preambles.	Persistence, bias to action, verification, parallel batch reads, concise final reporting.	Upfront plan chatter in rollout; incorrect <code>phase</code> handling when preambles are required.	Best for API-side customized coding agents; keep harness disciplined. ²⁷
GPT-5.2	More concise and disciplined than prior versions, but still prompt-sensitive; web research behavior is more steerable.	Pin <code>reasoning_effort</code> , define research bar, output shape, coverage expectations, and re-run evals after each change.	Migrating prompt and model simultaneously.	Make one change at a time and measure each step. ²⁸
GPT-5.1	Added <code>none</code> reasoning mode; stronger steerability in personality, tone, and formatting.	Completeness/persistence, explicit verbosity and formatting, low-latency mode where appropriate.	Assuming defaults preserve earlier behavior.	Good bridge from GPT-4.1 or GPT-5 for low-latency production use. ²⁹

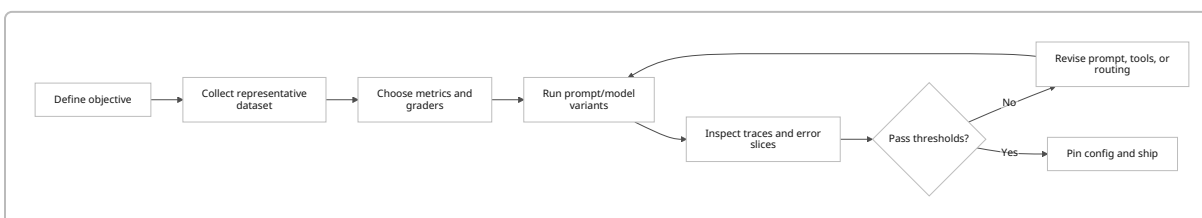
Model or family	What changed for prompting	What to emphasize	What to avoid	Practical recommendation
GPT-5 and GPT-4.1	GPT-4.1 is more literal and benefits from explicit instructions, examples, and long-context organization; step-by-step prompting can help.	Clear instructions, examples, long-context placement, XML delimiters, conflict cleanup.	Conflicting examples/rules; JSON-style document delimiters for long context.	Still relevant as a fallback mental model for instruction-sensitive tasks. ³⁰

One broader model-selection principle is worth keeping explicit: OpenAI’s own docs distinguish **reasoning models** from **GPT models** and note that GPT models are faster and more cost-efficient but benefit from more explicit instructions, while reasoning models are better for ambiguous, multi-step planning tasks. In practice, that means you should not try to turn every GPT prompt into a giant algorithm; sometimes the right answer is to switch model class or increase reasoning effort instead. ³¹

Community sentiment broadly reinforces the official guidance here. Hacker News and Reddit discussions repeatedly report that GPT-5/Codex tends to reward **specific upfront instructions**, while GPT-5.4+ reduced some of the awkwardness users felt in earlier GPT-5.x prompting. The OpenAI developer community also repeatedly emphasizes **tests as a behavioral constraint** and **small, reversible task loops** for Codex. These signals are useful, but lower-confidence than official docs or measured evals. ³²

Evaluation and debugging

OpenAI’s current eval guidance is straightforward: **evaluate early and often, design task-specific tests, log everything, automate scoring where possible, and calibrate automated metrics against human judgment**. The standard eval workflow is: define the objective, collect a representative dataset, define metrics, and compare runs over time. ³³



That loop is a synthesis of OpenAI’s evaluation best-practice docs and agent-eval workflow guidance. ³⁴

Evaluation method	Best for	Typical metrics or checks	Main caveats	Evidence
Programmatic metrics	Deterministic or semi-deterministic tasks	Exact match, string match, ROUGE/BLEU, function-call accuracy, executable evals	Can miss nuance or reward shallow matches	35
Executable checks	Coding, SQL, transformations, structured tasks	Unit tests, lint, compile checks, SQL execution, schema validation	Requires good fixtures and isolated environments	36
Human review	High-stakes judgments, style, nuanced helpfulness	Blinded ranking, numeric grading, pass/fail thresholds, consensus voting	Slow and expensive; humans disagree	37
LLM-as-judge / model graders	Scalable quality assessment	Pairwise comparison, single-answer grading, reference-guided grading	Position bias and verbosity bias; must calibrate to humans	38
Trace grading	Agents and Codex workflows	Tool choice, handoffs, guardrail violations, retry behavior, end-to-end workflow quality	Only useful if you already capture rich traces	39

OpenAI's official graders support at least three especially useful grader types for prompt work: `string_check` for exact pass/fail scoring, `text_similarity` for approximate or semantically tolerant matches using metrics such as fuzzy match, BLEU, METEOR, cosine, and ROUGE, and `score_model` or **Python graders** when you need semantic judgment or custom logic. [40](#)

The most common prompt failure modes are surprisingly mundane:

Failure mode	Typical cause	Fast mitigation
Conflicting instructions or examples	Different rules point in different directions; later prompt sections override earlier ones in practice.	Consolidate rules, align examples with rules, and place decisive instructions near the end of the controlling section. 41
Hallucinated or null tool calls	Over-absolute rules like "always call a tool first."	Make tool use conditional: call the tool when enough information is available; otherwise ask for the missing field or stop. 42
Robotic phrasing	Over-reliance on sample phrases.	Use fewer canned phrases and explicitly allow variation. 43

Failure mode	Typical cause	Fast mitigation
Format drift or extra prose	Output contract not explicit enough.	Use section headers or Structured Outputs; add examples when necessary. ⁴⁴
Long-context retrieval misses	Poor prompt organization or low-signal delimiters.	Put instructions above context, or both before and after in very long prompts; prefer XML or strong custom delimiters over JSON-like document packing. ⁴⁵
Codex stops early	Unnecessary preambles or incorrect <code>phase</code> handling in <code>gpt-5.3-codex</code> flows.	Remove rollout narration, or implement phase-aware handling correctly. ⁴⁶
Prompt bloat after migration	Carrying forward old prompt stacks into newer GPT-5.x models.	Reset to a shorter outcome-first baseline, then re-add only the constraints your evals prove are needed. ⁴⁷
Degradation after long sessions	Context growth, compaction drift, or omnibus skills/policies.	Split work into smaller missions, use focused skills, preserve tests as done criteria, and reopen a fresh thread when context quality decays. ⁴⁸

Tooling and workflows

The official OpenAI stack already covers most of the prompt lifecycle: **Responses API for generation, Structured Outputs for schema guarantees, hosted tools for retrieval and research, Agents SDK for orchestration and traces, graders/evals for measurement, and Codex SDK + skills for coding workflows.** ⁴⁹

Tool or workflow	What it is best for	Recommendation	Caveat	Evidence
Responses API + <code>instructions</code> / <code>developer</code> messages	Core prompt execution	Use for nearly all new prompt work; keep behavior instructions out of raw user input when possible.	<code>instructions</code> apply only to the current response call unless persisted another way.	⁵⁰
Structured Outputs with Pydantic or Zod	Stable machine-readable artifacts	Prefer for JSON-heavy production paths.	Not ideal when freeform creative prose is the product.	⁵¹
Web search and file search	Grounding with current or proprietary information	Prefer retrieval over assumptions when facts matter.	Still requires clear source-use rules and citation expectations.	⁵²

Tool or workflow	What it is best for	Recommendation	Caveat	Evidence
Prompt caching	Reducing cost and latency on repeated prompt prefixes	Put static instructions/examples first, variable content last.	Cache hits require exact prefix matches.	53
Agents SDK traces	Debugging agent behavior before formal evals	Inspect traces as soon as the first workflow runs.	Low observability makes later debugging much harder.	54
Prompt optimizer	Assisted prompt revision against a dataset	Useful during transition, especially when paired with grader results and annotations.	Dataset-backed optimizer is being deprecated with the Evals platform; always manually review outputs before shipping.	55
Codex SDK	CI/CD, internal coding agents, app integration	Use when you need thread continuity and programmatic control over the same Codex workflows you use interactively.	Server-side Node environment required for the SDK flow documented today.	56
Codex skills	Reusable, recurring engineering workflows	Use small, role-scoped skills with clean descriptions and trigger words.	Large omnibus skills can become context-heavy and slow.	22
Prompts-as-code in Git	Reviewability and rollback	Store prompts in code modules with typed inputs, tests, fixtures, PR review, and feature flags.	More setup than ad hoc prompting, but far safer in production.	16

A pragmatic modern workflow looks like this: define a typed prompt builder in code; add a schema if the output is consumed by software; connect retrieval only where needed; run a fixed eval set locally or in CI; inspect traces for failures; then gate deployment with a small scorecard. That pattern is almost exactly what OpenAI calls for in its prompting, production, and agent-evaluation docs. ⁵⁷

For teams that want a broader ecosystem stack beyond OpenAI-native tools, the most credible add-ons are currently **DSPy** for programmatic prompt optimization and modular LM workflows, **LangSmith** for evaluation and observability, and **promptfoo** for CI-friendly prompt testing and red teaming. These are not

replacements for good prompt design, but they are strong complements when your prompt surface becomes too large to manage manually. ⁵⁸

Security, safety, and alignment

The security baseline for prompt design should start from **instruction hierarchy**. OpenAI’s public work on instruction hierarchy frames trusted instruction order as **system > developer > user > tool**, and explicitly ties this to safety steerability and prompt-injection robustness. The practical implication is simple: **never treat retrieved text, web pages, or tool outputs as authoritative instructions**. Treat them as data. ⁵⁹

For agent and Codex workflows, prompt injection is not theoretical; OpenAI explicitly calls it a common and dangerous attack class. Their guidance recommends using **structured outputs between nodes**, limiting opportunities for free-form instruction smuggling, and putting human approvals and explicit consent around sensitive or destructive operations. In computer-use contexts, OpenAI also says the model should stop and escalate when it sees suspicious instructions or content that appears to be a prompt injection attempt. ⁶⁰

Good security-oriented prompt design therefore looks like this: constrain the input and output surface; prefer validated/closed-ended user inputs when feasible; use schemas between components; minimize what downstream tools can see; confirm before data exposure or irreversible actions; and log enough detail for audits and incident review. OpenAI’s safety best practices also recommend limiting user input length and output tokens to reduce prompt-injection and misuse surface area. ⁶¹

For Codex in particular, remember that model capability does not reduce the need for guardrails. OpenAI’s Codex safety docs note that GPT-5.3-Codex is treated as a **high cybersecurity capability** model with additional safeguards, and Codex configuration docs warn that even cached web search results should still be treated as untrusted. That means your prompt should specify when to stop, when to ask for confirmation, and when to refuse tool execution despite apparently relevant retrieved content. ⁶²

Prioritized checklist and sources

A useful prompt-design checklist is less about clever wording and more about sequencing the work correctly.

Priority	Checklist item	Why it matters
P0	Choose the right model and pin the version / reasoning effort before changing the prompt.	Otherwise you cannot tell whether gains or regressions came from the model or the prompt. ⁶³
P0	Write a short outcome-first contract : goal, success criteria, constraints, output, stop rule.	This is the highest-leverage pattern in the latest GPT guidance. ⁸
P0	If the output feeds software, use a schema .	Structured Outputs simplify prompting and cut format retries. ⁵¹

Priority	Checklist item	Why it matters
P0	Build a small eval set before expanding the prompt.	OpenAI explicitly recommends eval-driven development and warns against vibe-based iteration. ⁶⁴
P1	Add few-shot examples only where baseline behavior is unstable.	Examples are powerful, but they cost tokens and can overfit phrasing. ¹¹
P1	Add retrieval instead of general prompt bulk when facts matter.	Grounding beats instruction bloat for factual reliability. ⁶⁵
P1	For Codex, define autonomy, verification, and scope boundaries .	This is the difference between “helpful demo” and “reliable coding agent.” ⁶⁶
P1	Inspect traces before over-tuning the prompt.	Many failures are tool, routing, or approval failures rather than wording failures. ⁶⁷
P1	Keep static prompt prefixes stable for prompt caching .	This materially reduces cost and latency in repeated production traffic. ⁶⁸
P2	Use skills , meta-prompting, or prompt optimization for repeated workflows.	Helpful for scale, but only after you already have tests and representative data. ⁶⁹
P2	Introduce external tooling like DSPy, LangSmith, or promptfoo if prompt complexity outgrows ad hoc practice.	These help systematize optimization, observability, and CI gating. ⁷⁰

Prioritized source stack

Primary official sources

- OpenAI **Prompt guidance** for GPT-5.5, GPT-5.4, GPT-5.2, GPT-5.1, GPT-5, GPT-4.1, and GPT-5.3 Codex behavior changes. ⁷¹
- OpenAI **Prompt engineering** and **Prompting** guides for message roles, few-shot usage, prompts-as-code, Markdown/XML structure, and context/RAG patterns. ⁷²
- OpenAI **Models** and **Codex models** pages for current availability and product/API differences. ⁷³
- OpenAI **Evaluation best practices**, **Graders**, **Trace grading**, and **Evaluate agent workflows** docs for testing methodology. ⁷⁴
- OpenAI **Structured Outputs**, **Web search**, **File search**, **Prompt caching**, **Agents SDK**, **Codex SDK**, and **Skills** docs for implementation workflow. ⁷⁵
- OpenAI **Safety in building agents**, **Safety best practices**, **instruction hierarchy** work, and **computer use** safety guidance. ⁷⁶

Technical and research sources

- LangChain on **prompt optimization** and **context engineering**. Useful for external workflow framing and optimization strategy. ⁷⁷

- Academic papers on **chain-of-thought**, **self-consistency**, **least-to-most prompting**, and large prompt-engineering surveys. Useful for understanding when decomposition and multi-sample reasoning help, and where those techniques sit in the literature. ⁷⁸

Community sources with lower evidentiary weight

- OpenAI Developer Community discussion on **tests as done criteria**, **small reversible loops**, and **skills vs. token budget**. ⁷⁹
- Hacker News and Reddit discussions indicating that newer GPT/Codex versions reward **clear upfront specificity**, while users often experience prompt behavior changes across versions. Useful as anecdotal triangulation, not as ground truth. ⁸⁰

¹ ⁶ ⁸ ⁹ ¹⁴ ²⁴ ²⁵ ²⁶ ²⁸ ²⁹ ⁴⁷ ⁷¹ **Prompt guidance | OpenAI API**
<https://developers.openai.com/api/docs/guides/prompt-guidance>

² ⁴ ⁷³ **Models | OpenAI API**
<https://developers.openai.com/api/docs/models>

³ ¹⁵ ⁷⁸ **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models**
https://arxiv.org/abs/2201.11903?utm_source=chatgpt.com

⁵ **Codex Models**
https://developers.openai.com/codex/models?utm_source=chatgpt.com

⁷ ⁴¹ ⁴² ⁴³ ⁴⁴ ⁴⁵ **GPT-4.1 Prompting Guide**
https://developers.openai.com/cookbook/examples/gpt4-1_prompting_guide

¹⁰ ⁵¹ ⁷⁵ **Structured model outputs | OpenAI API**
<https://developers.openai.com/api/docs/guides/structured-outputs>

¹¹ ¹² ¹³ ²³ ³¹ ⁴⁹ ⁵⁰ **Prompt engineering | OpenAI API**
<https://developers.openai.com/api/docs/guides/prompt-engineering>

¹⁶ ⁵⁷ ⁷² **Prompting | OpenAI API**
<https://developers.openai.com/api/docs/guides/prompting>

¹⁷ ¹⁸ ¹⁹ ²¹ ²⁷ ⁴⁶ ⁶⁶ **Codex Prompting Guide**
https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide

²⁰ **Workflows – Codex | OpenAI Developers**
<https://developers.openai.com/codex/workflows>

²² ⁶⁹ **Agent Skills – Codex | OpenAI Developers**
<https://developers.openai.com/codex/skills>

³⁰ **GPT-4.1 Prompting Guide**
https://developers.openai.com/cookbook/examples/gpt4-1_prompting_guide?utm_source=chatgpt.com

³² ⁸⁰ **GPT-5-Codex | Hacker News**
<https://news.ycombinator.com/item?id=45252301>

³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁸ ⁶⁴ ⁷⁴ **Evaluation best practices | OpenAI API**
<https://developers.openai.com/api/docs/guides/evaluation-best-practices>

39 67 Evaluate agent workflows | OpenAI API

<https://developers.openai.com/api/docs/guides/agent-evals>

40 Graders | OpenAI API

<https://developers.openai.com/api/docs/guides/graders>

48 Creating and Prompting Codex Skills - Prompting - OpenAI Developer Community

<https://community.openai.com/t/creating-and-prompting-codex-skills/1376382>

52 65 File search | OpenAI API

<https://developers.openai.com/api/docs/guides/tools-file-search>

53 68 Prompt caching | OpenAI API

<https://developers.openai.com/api/docs/guides/prompt-caching>

54 Integrations and observability | OpenAI API

<https://developers.openai.com/api/docs/guides/agents/integrations-observability>

55 Prompt optimizer | OpenAI API

<https://developers.openai.com/api/docs/guides/prompt-optimizer>

56 SDK – Codex | OpenAI Developers

<https://developers.openai.com/codex/sdk>

58 70 DSPy: The framework for programming—not prompting— ...

https://github.com/stanfordnlp/dspy?utm_source=chatgpt.com

59 Improving instruction hierarchy in frontier LLMs | OpenAI

<https://openai.com/index/instruction-hierarchy-challenge/>

60 76 Safety in building agents | OpenAI API

<https://developers.openai.com/api/docs/guides/agent-builder-safety>

61 Safety best practices | OpenAI API

<https://developers.openai.com/api/docs/guides/safety-best-practices>

62 Cyber Safety – Codex

https://developers.openai.com/codex/concepts/cyber-safety?utm_source=chatgpt.com

63 GPT-5.2 Prompting Guide

https://developers.openai.com/cookbook/examples/gpt-5/gpt-5-2_prompting_guide

77 Exploring Prompt Optimization

<https://www.langchain.com/blog/exploring-prompt-optimization>

79 Tips and Tricks for using Codex - Codex - OpenAI Developer Community

<https://community.openai.com/t/tips-and-tricks-for-using-codex/1373143>